Mark	

Team name	*A10*
Homework number	*HOMEWORK 03*
Due date	09/10/2026

Contribution	NO	Partial	Full
Carbone Alessandro			*X*
Casareale Alberto			*X*
Colombi Federico			*X*
Lorusso Luca			*X*
Martorano Gabriele			*X*
Notes: Everyone contributed to understand the logic and implemented the code			

Project name	Play a song		
Not done	Partially done (major problems)	Partially done (minor problems)	Completed
			X

Explanation:

Part 1: Hardware Routing and Configuration

To achieve the goal of playing a melody upon detecting a sound, we first consulted the 'Hands-on Lab Schematics.pdf' and the 'NUCLEO user-manual.pdf' to identify the correct hardware routing. The schematics confirmed that the microphone's output signal is physically connected to the PA8 pin of the microcontroller, while the speaker is connected to the PA9 pin. In the STM32CubeMX software, we configured the PA8 pin as an external interrupt (GPIO_EXTI8) and set its trigger detection to respond to the rising edge. For the speaker, we configured the PA9 pin as the output of the PWM channel 2 of Timer 1 (TIM1_CH2). To ensure the microcontroller processes the microphone signals, we enabled the corresponding interrupt (EXTI9_5) by checking the enable box in the NVIC settings, and we also enabled the global interrupt for Timer 2 (TIM2 global interrupt), which we will use as a time base.

Part 2: Code Generation and Verification

After generating the project code, we configured the timers in the main.c file. For Timer 1 (used for PWM generation), we set the prescaler (PSC) to 100-1 and the counter period (ARR) to 3206-1, which corresponds to the initial period of the 'DO' note. For Timer 2 (used as a time base), we set the prescaler (PSC) to 84-1 and the counter period (ARR) to 1000-1. This configuration, assuming an 84 MHz clock, sets the base frequency for Timer 2 to 1 kHz (1 ms ticks).

Part 3: Implementation of Functional Logic

Finally, we implemented the functional logic. We defined an array of 16-bit integers (notes) of size 64 to store the period values for the melody, using macro definitions for the musical note frequencies (e.g., SOL, LA, FA). We also set a single variable (time = 200) for the uniform duration of each note in milliseconds.

We used the external interrupt callback function HAL_GPIO_EXTI_Callback to detect the sound from the microphone. As a specific design choice to improve the system's reliability, we implemented a 1-second cooldown mechanism using the HAL_GetTick() function. This prevents the microphone from accidentally re-triggering the melody by picking up the speaker's residual echoes. When a sound is detected, and provided the system is not already playing and the 1000 ms cooldown has elapsed, this function immediately loads the first note's value into Timer 1 (__HAL_TIM_SET_AUTORELOAD) and sets the duty cycle to 50% (__HAL_TIM_SET_COMPARE to notes[note] / 2). It then resets the counters of both Timer 1 and Timer 2 (__HAL_TIM_SET_COUNTER) to ensure a clean start, starts the PWM generation, and sets a flag (play = 1) to indicate that the melody should be played.

The melody progression is handled by the Timer 2 interrupt callback function HAL_TIM_PeriodElapsedCallback. Every time Timer 2 triggers an interrupt (every millisecond), a counter (tim) is incremented. When this counter reaches the specified duration of the current note (time), the code increments the note index to move to the next note. If the end of the array is reached (size = 64), the PWM generation is stopped (HAL_TIM_PWM_Stop), silencing the speaker, and the last_play_end variable is updated with the current system tick to start the cooldown timer. Finally, the playback flag is reset to wait for the next sound trigger. Otherwise, it updates the ARR and Compare registers of Timer 1 to change the frequency and duty cycle for the next note in the array.

Professor comments: